ДОМАШНЕЕ ЗАДАНИЕ

1. Выбор веб-приложения и определение используемых протоколов

Выбранное приложение: Twitch (платформа прямой трансляции видео, Amazon).

Twitch — один из крупнейших сервисов live-стриминга в мире. Пользователи ежедневно смотрят трансляции игр, киберспорта, творческого контента и общаются в чате. Приложение предъявляет высокие требования к низкой задержке, масштабируемости доставки и устойчивости к потерям пакетов.

Транспортные протоколы

Основной транспортный протокол для массовой доставки зрителям: TCP (через HTTP/2 и HTTP/1.1).

Классическая доставка видео на Twitch построена на HLS (HTTP Live Streaming). Зритель загружает манифест и медиасегменты по HTTP. Под этим работает TCP. Этот подход хорошо масштабируется через CDN, кэшируется и надёжно проходит через большинство сетей.

Современные и экспериментальные направления: QUIC (поверх UDP) и WebRTC (поверх UDP).

Twitch активно исследует и внедряет низколатентные решения. Для ultra-low latency применяются подходы на базе WebRTC. Также компания участвует в разработке и тестировании Media over QUIC (MoQ) — протокола поверх QUIC, который сочетает низкую задержку с хорошей масштабируемостью. HTTP/3 (QUIC) постепенно используется для части трафика и как основа будущих решений.

Прикладные протоколы

HLS (HTTP Live Streaming) — основной протокол доставки видео зрителям. Работает поверх HTTP/1.1 или HTTP/2 (TCP).

RTMP — классический протокол для приёма (ingest) потока от стримера. Работает поверх TCP.

WebRTC — используется для режимов с особо низкой задержкой и интерактивных сценариев.

HTTP/2 и HTTP/3 — транспорт для манифестов, сегментов и API. HTTP/3 (поверх QUIC) применяется всё шире.

WebSocket — для чата и real-time событий (поверх TCP).

Итоговый типичный стек для зрителя: HLS → HTTP/2 (или HTTP/3) → TCP (или QUIC/UDP) → IP. Для ingest: RTMP → TCP.

2. Обоснование выбора стека и альтернативы

Почему выбран именно такой стек

Twitch решает две разные задачи:

- **Ingest (приём потока от стримера)** — нужна относительно низкая задержка и надёжность. RTMP поверх TCP исторически стал стандартом, потому что прост в реализации и хорошо работает за NAT.
- **Distribution (доставка миллионам зрителей)** — нужна максимальная масштабируемость, кэширование на CDN и совместимость с браузерами и устройствами. HLS поверх HTTP/TCP идеально подходит: сегменты кэшируются, протокол поддерживается практически везде, легко применять adaptive bitrate.

Преимущества текущего подхода:

- Отличная масштабируемость через обычные HTTP-CDN.
- Высокая совместимость (браузеры, Smart TV, мобильные приложения).
- Простая реализация adaptive streaming.
- Надёжная работа в сетях, где UDP блокируется.

Возможные альтернативы и почему они менее подходят (или дополняют)

1. Чистый WebRTC для всей доставки

Даёт очень низкую задержку, но плохо масштабируется на миллионы зрителей (каждый зритель — отдельное медиасоединение или сложная mesh/SFU-архитектура). Выше стоимость инфраструктуры и сложнее кэширование. Поэтому WebRTC на Twitch используется точечно, а не как основной способ массовой доставки.

2. Полный переход только на UDP без надёжности

Минимальная задержка, но требует собственной реализации повторных передач, упорядочивания и управления перегрузкой. Кроме того, UDP часто блокируется или ограничивается в корпоративных и мобильных сетях.

3. Media over QUIC (MoQ) и HTTP/3

Именно в эту сторону движется индустрия и сам Twitch. MoQ поверх QUIC обещает сочетать низкую задержку WebRTC с масштабируемостью HTTP. HTTP/3 уже даёт преимущества (меньше Head-of-Line Blocking, быстрый

handshake, connection migration). Полный переход — вопрос времени и зрелости экосистемы.

4. SRT / RIST

Отлично подходят для профессиональной contribution (доставка сигнала от источника до платформы), но не предназначены для массового потребления конечными зрителями через браузеры.

Текущий гибридный стек (HLS/TCP + точечный WebRTC + движение в сторону QUIC/MoQ) — прагматичный выбор, балансирующий совместимость, стоимость и задержку.

3. Преимущества и ограничения перехода на QUIC/HTTP/3

Twitch уже экспериментирует с QUIC и Media over QUIC. Рассмотрим, какие преимущества дал бы более полный переход на QUIC/HTTP/3 (и дальше на MoQ), а также какие ограничения могут возникнуть.

Преимущества

- **Снижение задержки.** 1-RTT/0-RTT handshake и отсутствие Head-of-Line Blocking позволяют быстрее доставлять сегменты и уменьшить end-to-end latency live-трансляций.
- **Лучшая устойчивость к потерям.** Потеря пакета в одном потоке не блокирует остальные (аудио, видео разных качеств, чат-метаданные).
- **Мобильность соединения.** Connection ID позволяет сохранять сессию при переходе с Wi-Fi на мобильную сеть без перезапуска потока — важно для мобильных зрителей.
- **Встроенная безопасность.** TLS 1.3 интегрирован в протокол, метаданные транспорта шифруются.
- **Путь к Media over QUIC.** QUIC становится фундаментом для новых протоколов (MoQ), которые сочетают низкую задержку и хорошую масштабируемость — именно то, что нужно live-платформам.

Возможные ограничения и проблемы

- **Блокировка или ограничение UDP.** В части сетей UDP фильтруется. Требуется надёжный fallback на TCP/HTTP/2, иначе часть аудитории потеряет качество или доступ.
- **Сложность мониторинга и отладки.** Зашифрованные заголовки QUIC затрудняют анализ трафика операторами связи и традиционными DPI-системами.
- **Нагрузка на инфраструктуру.** Реализация QUIC в userspace и обязательное шифрование повышают потребление CPU на edge-серверах и CDN по сравнению с классическим TCP.

- **Экосистема и совместимость.** Не все CDN, плееры и промежуточное оборудование одинаково хорошо поддерживают HTTP/3 и особенно экспериментальные протоколы вроде MoQ. Нужен период сосуществования со старым стеком.
- **Сложность миграции.** Перевод всей цепочки (ingest → обработка → origin → CDN → плеер) на новый транспорт — большой инженерный проект с рисками регрессий.

Для Twitch переход на QUIC/HTTP/3 и далее на MoQ выглядит стратегически правильным, но требует поэтапного внедрения с сохранением TCP-fallback.

4. Пример приложения, для которого UDP подходит лучше, чем TCP

Пример: онлайн-игра в реальном времени (например, шутер от первого лица — Counter-Strike 2, Valorant, Fortnite) или система голосовой/видеосвязи в реальном времени (VoIP, WebRTC-звонки).

Почему UDP предпочтительнее TCP

В таких приложениях критически важны:

- **Минимальная задержка (latency).** Игрок должен видеть действия противников и отправлять свои команды с задержкой в десятки миллисекунд. Даже 50–100 мс уже ощутимы.
- **Допустимость потери отдельных пакетов.** Потеря одного обновления позиции персонажа или одного голосового фрейма обычно не критична. Гораздо хуже — задержка из-за повторной передачи.
- **Нежелательность Head-of-Line Blocking.** Если один пакет потерян, TCP будет ждать его повторной передачи и задержит все последующие данные. В игре это приводит к «фризам» и несправедливому преимуществу оппонентов.

UDP позволяет:

- Отправлять данные сразу, без установления соединения и подтверждений.
- Приложению самостоятельно решать, какие данные важны (и нужна ли повторная передача), а какие можно просто отбросить.
- Реализовывать собственные механизмы компенсации потерь (интерполяция, предсказание движения, FEC — forward error correction).

Как решается проблема надёжности

Приложения поверх UDP обычно реализуют:

- Собственные номера последовательностей и подтверждения только для критичных сообщений (например, факт попадания пули).
- Периодическую отправку полного состояния мира (snapshot), чтобы клиент мог восстановиться после серии потерь.
- Алгоритмы предсказания и сглаживания на стороне клиента.

В современных системах часто используют гибридный подход: UDP для игрового состояния и голоса, а TCP/QUIC — для загрузки ассетов, авторизации, чата и других не критичных по задержке данных.

Вывод: для приложений реального времени, где важнее свежесть данных, чем гарантированная доставка каждого байта, UDP (часто в сочетании с собственными механизмами надёжности или с QUIC Datagram) является более подходящим выбором, чем классический TCP.

Заключение

Выбор транспортного протокола — это архитектурное решение, которое напрямую влияет на пользовательский опыт, масштабируемость и стоимость инфраструктуры. Twitch демонстрирует прагматичный гибридный подход: HLS поверх TCP для массовой доставки и эксперименты с WebRTC и QUIC/MoQ для снижения задержки. В то же время для задач с жёсткими требованиями к задержке (онлайн-игры, VoIP) по-прежнему часто предпочтительнее UDP с прикладной логикой надёжности. Универсального «лучшего» протокола не существует — всё зависит от конкретных требований приложения.